

---

# Beyond the Token Bottleneck: Latent Communication Has a Receiver-Consumption Bottleneck

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

1       Recent work on latent communication argues that natural language is a low-  
2       bandwidth bottleneck between LLM agents. We argue that this view is correct  
3       but incomplete. A communication channel is defined not by the richness of the  
4       sender’s state, but by the receiver’s ability to use that state through its own com-  
5       putation. We call this gap the *receiver-consumption bottleneck*. Latent commu-  
6       nication should therefore be studied as receiver-consumption design rather than  
7       sender-representation transfer. Existing protocols already instantiate different con-  
8       sumption semantics: expected embeddings as soft inputs, final hidden states as  
9       adapter-mediated messages, residual hidden states as frozen-suffix continuations,  
10      and KV caches as attention memory. These states are not interchangeable mes-  
11      sages. Controlled diagnostics show that residual states are exact only under the  
12      correct continuation contract, cross-model transfer is not dimensional alignment,  
13      representation alignment need not improve behavior, and cut-layer choice changes  
14      readability. Task gains alone also do not establish sample-specific latent com-  
15      munication. These failures share a structural origin: external state must enter  
16      computation paths the receiver can validate. This predicts that architectural read  
17      interfaces, such as input-side and attention-side interfaces, should transfer better  
18      across models than raw residual injection. Future work should report receiver-  
19      consumption contracts and validate receiver readability under perturbations to layer,  
20      model, adapter, position policy, and task distribution. The goal is not just to send  
21      richer latent states, but to understand when receivers can actually consume them.

## 22   1 Introduction

23   Recent work on latent communication starts from a simple observation. When LLM agents commu-  
24   nicate through natural language, they must compress internal computation into discrete tokens and  
25   rely on another model to reconstruct useful state from text. This token interface is expressive and  
26   human-readable, but it can also be narrow, lossy, verbose, and expensive. These limitations motivate  
27   a growing body of work on communication below natural language.

28   Existing work instantiates this idea in several forms. CIPHER-style protocols let agents communicate  
29   through expected embeddings rather than sampled tokens [24]. Activation communication transfers  
30   intermediate activations between language model agents [26]. Interlat studies agent communica-  
31   tion entirely in latent space through adapter-mediated hidden-state messages [5]. LatentMAS and  
32   thought-communication methods introduce latent thoughts or working-memory states for multi-agent  
33   collaboration [34, 35]. KV-cache communication methods selectively share, reuse, project, or fuse  
34   attention caches across contexts and models [7, 27, 33]. A related line of work studies continuous rea-  
35   soning states, pause tokens, soft chains of thought, test-time soft reasoning, and supervised thinking  
36   states [1, 12, 14, 30, 31]. Together, these works show that useful computation need not be represented  
37   only as natural-language tokens.

# Model Communication and the Information Utility Gap

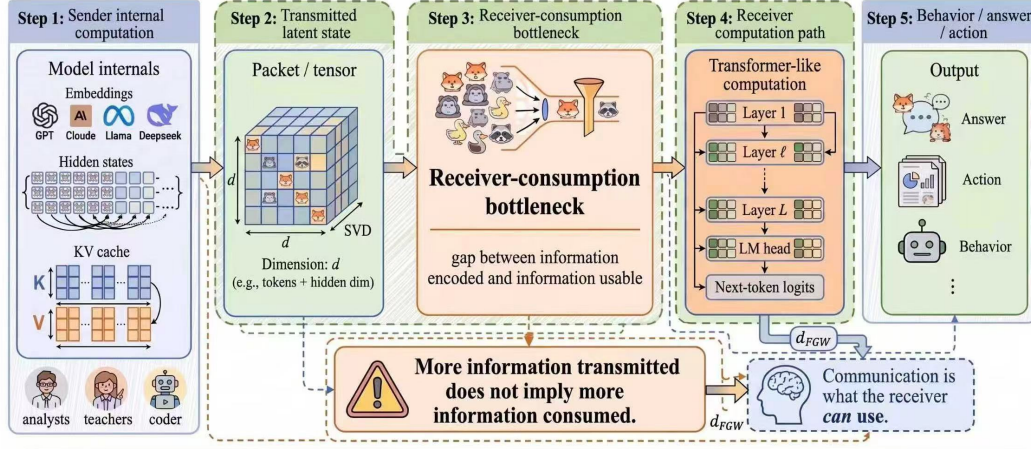


Figure 1: The receiver consumption bottleneck in latent communication. A sender may expose a rich latent state, but only the part that the receiver can consume through its own computation path can affect answers, actions, or downstream behavior.

We agree with this motivation, but argue that it is incomplete. Removing the token bottleneck does not by itself establish a communication channel. A high-dimensional latent state may encode task-relevant information for the sender, yet that information matters only if the receiver can use it through its own computation. The relevant question is therefore not only what state is transmitted, but also how the receiver consumes that state.

We call this gap the receiver consumption bottleneck. It is the gap between information encoded in a transmitted latent state and information that can influence the receiver’s actual computation and behavior. This bottleneck does not replace the token bottleneck. It becomes visible once communication moves below natural language. A latent state can be rich, high-dimensional, and probe-readable while still failing to affect the receiver’s answer. Conversely, a lower-dimensional state can be useful if it enters a receiver interface that is trained or structured to consume it.

Figure 1 illustrates this view. Increasing the size, dimension, or apparent richness of a transmitted latent state does not guarantee that more information becomes usable by the receiver. Only the portion that enters a compatible receiver computation can affect downstream behavior.

This distinction is important because current latent communication methods differ not only in the tensors they transmit, but also in the receiver interfaces they assume. Expected embeddings are consumed as input-side soft tokens [24]. Intermediate activations are consumed by resuming a forward pass from a compatible continuation state [26]. Final hidden states may require an adapter-mediated reader [5]. Latent thoughts and working memories are consumed recurrently or through attention-side memory mechanisms [34, 35]. KV caches are consumed as attention memory, often under layer, offset, or projection constraints [7, 27, 33]. These states are all latent, but they are not interchangeable communication objects.

Our claim is not that prior work ignores the receiver. Many successful protocols can be understood as concrete solutions to receiver consumption. The limitation is that receiver consumption is usually analyzed within a specific protocol rather than treated as a general object of comparison. As a result, it is often unclear whether a reported gain depends on the state type, the receiver model, the layer, the adapter, the position policy, or the task distribution.

We argue that latent communication should be evaluated through receiver consumption contracts. A contract specifies what state is transmitted, where it enters the receiver, which computation consumes it, and under what conditions the claim is expected to hold. Such contracts make it possible to compare protocols without treating embeddings, residual states, hidden states, latent thoughts, and KV caches as interchangeable messages.

We further argue that latent communication should be validated through receiver readability. A latent channel should be compared against a direct-information baseline that estimates what the receiver could do with accessible task information, and against a source-denied baseline that tests whether the receiver can already solve the task without the latent state. Sample-specific controls should test whether the transmitted state matters for the particular source instance. Robustness checks should vary layer, receiver model, adapter, position policy, and task distribution.

This paper is a position paper rather than a new adapter method or a full survey. The experiments below are diagnostic probes for the receiver-consumption view. They use Qwen-family decoder-only models as controlled testbeds [32]. The diagnostics show that residual hidden states are exact only under the correct continuation contract, cross-model transfer cannot be explained by dimensional alignment alone, representation alignment need not improve downstream behavior, and cut-layer choice changes receiver readability.

The diagnostics and the structural analysis are mutually reinforcing. Section 4 presents failure modes that, under the receiver-consumption view, follow from a single architectural fact: latent states are usable only through receiver computation paths that can consume them. Appendix B makes this explicit for cross-model consumption, predicting why receiver-owned read interfaces such as input-side or attention-side consumption should scale better than raw residual injection.

We make six contributions.

1. We reframe latent communication as a receiver-consumption problem rather than a sender-representation problem.
2. We introduce the receiver-consumption bottleneck as the gap between information encoded in a transmitted latent state and information usable by the receiver’s actual computation path.
3. We organize existing latent-communication protocols by receiver-consumption semantics, showing that embeddings, hidden states, KV caches, and latent thoughts are not interchangeable communication objects.
4. We provide controlled diagnostic probes showing that receiver consumption is sensitive to continuation, model compatibility, representation objectives, and cut layer.
5. We connect these diagnostics to a structural prediction: cross-model latent communication should scale best through receiver-owned architectural read interfaces.
6. We argue for receiver-consumption contracts, receiver-readability validation, and hybrid latent-text fallback as practical field norms for latent communication.

## 2 Sender Representations Are Not Communication Channels

A communication channel is not defined only by information present in the transmitted state. It is defined by whether another system can use that state to change its behavior. This matters for latent communication because the transmitted objects are often information-rich: expected embeddings may encode token-level beliefs [24], hidden states may contain task information [5, 26], KV caches may preserve context for attention-side reuse [7, 27, 33], and latent thoughts may carry intermediate reasoning state [34, 35]. Yet information present in a representation is not necessarily information available to the receiver.

We distinguish three notions.

**Probe readable.** An auxiliary classifier, decoder, or probe can extract information from the state.

**Aligned.** The state is close to another state under a representation metric, such as MSE, cosine similarity, or KL divergence over derived logits.

**Receiver readable.** The receiver’s actual computation path can consume the state and produce the intended downstream behavior.

These notions can diverge. A state may be probe readable but not receiver readable if the receiver lacks a mechanism to extract the relevant information. A state may align with a teacher representation but fail behaviorally, because later transformer layers are not generic decoders from hidden space. A high-dimensional state may also be ineffective when inserted at the wrong layer or under the wrong position policy. Similar issues arise in continuous reasoning and latent-state methods, where useful

Table 1: Receiver-side semantics of representative latent-communication protocols.

| Protocol       | State                                                          | Receiver consumption                                                          |
|----------------|----------------------------------------------------------------|-------------------------------------------------------------------------------|
| Soft-token     | Expected embeddings, soft tokens, vocabulary-belief embeddings | Input-side embedding or soft-token substitution [24, 30, 31]                  |
| Activation     | Intermediate activations                                       | Continuation from a combined intermediate activation state [26]               |
| Interlat-like  | Final-layer hidden states                                      | Adapter-mediated latent-message reading [5]                                   |
| LatentMAS-like | Latent thoughts and layer-wise KV working memory               | Recurrent latent state with attention-side working memory [1, 12, 14, 34, 35] |
| KV sharing     | Selected KV pairs or reusable KV caches                        | Attention-memory consumption with layer or offset alignment [27, 33]          |
| Cache-to-cache | Projected source-model KV cache                                | Target-cache projection, fusion, and layer gating [7]                         |

computation may appear as pause states, soft chains of thought, or supervised thinking states rather than ordinary tokens [1, 12, 14, 30, 31].

This motivates the central claim. A latent state becomes communicative only when a receiver can use it. A latent communication claim should therefore specify

(sender state, transport operator, receiver computation, validity scope, behavioral validation).

The sender state is the transmitted object, such as an expected embedding, hidden state, KV cache, compressed cache, or recurrent thought. The transport operator specifies how it is moved or inserted, such as prefix insertion, hidden replacement, cache fusion, side memory, or recurrent feedback. The receiver computation specifies what consumes it, such as input embedding consumption, frozen-suffix continuation, attention memory, adapter-mediated reading, or a learned cache fuser.

The remaining fields make the claim empirical. Validity scope states when the protocol is expected to hold, including assumptions about model identity, layer, adapter, position policy, and task distribution. Behavioral validation tests whether the receiver actually uses the state to improve downstream behavior. Without scope and validation, the description identifies a tensor pathway rather than a communication channel.

This formulation treats latent communication as a claim about how a receiver turns a transmitted state into behavior. It also separates failure modes: the state may lack the relevant information, the transport operator may deliver it to incompatible computation, the receiver may lack a learned reader, or the evaluation setting may lie outside scope. These are different scientific claims and should not be collapsed into aggregate accuracy.

### 3 Existing Latent Communication as Receiver Consumption Interfaces

Table 1 organizes representative latent communication protocols by how the receiver consumes the transmitted state. The goal is not to provide a full survey, but to make the receiver interface explicit. Across methods, the central design question is which part of the receiver’s computation is allowed to turn the transmitted state into useful behavior.

This interface-centered view is consistent with several adjacent lines of work. Prefix tuning and prompt tuning show that learned continuous vectors become useful when they enter a model through an input-side or prefix-side interface [20, 21]. Adapter and low-rank adaptation methods show that small learned modules can change how frozen model computation consumes task information [16, 17, 23]. Memory-augmented and retrieval-augmented transformers show that external information becomes useful through specified recurrence, attention, or retrieval interfaces [3, 4, 13]. Work on transformer memories and activation interventions further suggests that hidden states should be interpreted relative to their computational site rather than as free-standing semantic objects [10, 11, 22, 29].

Table 2: Diagnostic lessons for receiver consumption failures. Detailed protocols and numerical results are reported in Appendix A.

| Diagnostic           | Observation                                                                 | Receiver-consumption lesson                                                              |
|----------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Continuation         | Exact replay holds only under the correct next-layer continuation.          | A residual hidden state is a continuation state rather than a self-contained message.    |
| Cross-model transfer | Simple dimensional maps fail, and calibrated maps depend on layer pairing.  | Shape compatibility does not imply receiver compatibility.                               |
| Alignment objective  | Hidden-state alignment can improve while downstream behavior worsens.       | Representation reconstruction is an auxiliary diagnostic, not a communication guarantee. |
| Cut-layer sweep      | Early cuts are more readable than late cuts in the tested setting.          | Layer choice is part of the receiver-consumption claim.                                  |
| Task-gain ambiguity  | A task gain can arise without proving sample-specific latent communication. | Task improvement must be paired with bracketing and sample-specificity controls.         |

This organization suggests a different vocabulary for comparing methods. Latent communication is often described by the object being transmitted, such as an embedding, hidden state, KV cache, or latent thought. These labels are useful, but they do not fully specify the communication channel. The same class of tensor can have different meanings depending on whether the receiver treats it as an input substitute, an adapter-readable message, an attention memory, a cache-fusion object, or a continuation state.

The more useful unit of comparison is therefore the receiver consumption interface. This interface specifies where the state enters the receiver, what computation consumes it, and under what assumptions it remains valid. Under this view, many differences between existing methods are not implementation details. They define different claims about what the receiver can read and how that readability is preserved.

This perspective also reframes the field’s central difficulty. The problem is not only bandwidth, but portability of consumption. A latent channel that works for one receiver computation may fail when the checkpoint, layer, adapter, position policy, or task distribution changes. The next sections make this point concrete through receiver consumption contracts and receiver readability tests.

## 4 Five Ways Latent States Fail to Become Communication

We use controlled experiments as diagnostic probes for the receiver consumption bottleneck. The goal is not to propose a new communication method, but to isolate cases where an informative latent state fails to become usable by the receiver. This framing is related to prior work showing that representation similarity does not imply behavioral equivalence [18, 19, 25], and that internal states should be tested through causal or behavioral interventions [8, 11, 22, 28, 29]. Appendix A reports the full protocols and numerical results.

The diagnostics identify four receiver-computation failures and one validation failure. Residual states can fail when the continuation contract changes. Cross-model transfer can fail despite dimensional compatibility. Representation alignment can improve while downstream behavior worsens. Layer choice can determine whether the receiver can use a latent state. Finally, task gains can occur without proving sample-specific latent communication, a concern related to shortcut learning and distribution-specific effects [2, 9]. Table 2 summarizes these lessons.

**Continuation is a contract.** In same-checkpoint splicing, residual states can be cut and reattached exactly only when the original continuation conditions are preserved, including checkpoint, tokenizer, attention mask, position IDs, numerical mode, and next-layer suffix. Replaying, skipping, or moving across layers changes the computation that consumes the state. Under frozen-suffix continuation, a residual hidden state is therefore a continuation state rather than a standalone message.

Table 3: A receiver consumption contract defines the scope of a receiver-readability claim.

| Field            | Specifies                                                                                      | Purpose                                                                                                     |
|------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Endpoints        | State type, sender model, receiver model, tokenizer, and numerical mode                        | Defines the object being transmitted and the systems allowed to consume it.                                 |
| Consumption path | Consumption site, layer policy, position policy, attention policy, and transport operator      | Identifies the receiver computation that can read the transmitted state.                                    |
| Adaptation       | Adapter, projector, fuser, calibration data, and training distribution                         | Separates dimensional or geometric compatibility from learned receiver readability.                         |
| Claim scope      | Validity conditions, evaluation metrics, perturbation tests, audit policy, and fallback policy | States where the usability claim applies and when it should be rejected or replaced by text or tool traces. |

181 **Compatibility is more than shape.** Cross-model transfer shows that matching dimensionality is  
182 not sufficient. Zero padding, repeat padding, and embedding-space maps do not make hidden states  
183 directly consumable by a larger receiver. Calibrated maps can help, but depend on layer pairing. The  
184 relevant compatibility is computational, not merely geometric.

185 **Alignment is not consumption.** Representation alignment can be informative, but it does not  
186 establish receiver readability. In our diagnostics, hidden compensation can improve an alignment  
187 objective while worsening downstream behavior. This separates representation reconstruction from  
188 actual use by the receiver’s computation path.

189 **Layer is part of the claim.** In same-model dual-source latent-prefix communication, early cuts are  
190 more receiver-readable than late cuts in the tested setting. This does not imply that early layers are  
191 always better. It shows that insertion layer is part of the communication contract, not an incidental  
192 implementation detail.

193 **Task gain is not communication.** End-to-end task improvement shows that an intervention helped,  
194 but not what information was consumed. A gain may come from formatting, routing, learned priors,  
195 or distributional cues rather than source-specific latent content. A communication claim therefore  
196 needs bracketing and sample-specificity controls.

197 These failures share one source: a latent state affects behavior only through the receiver computation  
198 that consumes it. When that computation changes, the same state may stop being communicative  
199 even if it remains high-dimensional, probe-readable, or representation-aligned.

## 200 5 Receiver Consumption Contracts

201 The diagnostics motivate a reporting and design abstraction that we call a receiver consumption  
202 contract. A contract specifies the conditions under which a latent state is claimed to be usable by a  
203 receiver. It is not itself evidence that the channel works. Instead, it defines the boundary that makes a  
204 usability claim meaningful, testable, and falsifiable.

205 A full contract can instantiate these fields with additional detail, including the sender and receiver  
206 checkpoints, the source and receiver layers, the position and attention conventions, the adapter  
207 or cache fuser, the calibration distribution, and the audit or fallback policy. These details are not  
208 incidental implementation choices. They determine which receiver computation is being asked to  
209 turn a latent state into behavior.

210 The contract is not meant to impose a single protocol family. It can describe final-hidden-state mes-  
211 sages, latent working memory, cache fusion, activation communication, or residual-state continuation.  
212 Its purpose is to make the receiver side explicit across methods that otherwise appear to use unrelated  
213 latent objects.

Table 4: Three null explanations that receiver readability validation should rule out.

| Null explanation   | Test family        | Required evidence                                                                                                                                                    |
|--------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Latent not needed  | Bracketing         | The latent path improves over a source-denied baseline and is interpreted relative to a direct-information baseline.                                                 |
| Content irrelevant | Sample specificity | The gain depends on matching the transmitted state to the source instance. Shuffled, zero, constant, or distribution-matched latents should not preserve the effect. |
| Scope overclaimed  | Scope robustness   | Perturbations to layer, position policy, attention policy, adapter, receiver, or task family degrade behavior in a way consistent with the claimed scope.            |

214 It is useful to separate three levels of description.

**Protocol.** The procedure that produces, transmits, and consumes a latent state.

215 **Contract.** The assumptions under which the protocol’s usability claim is scoped.

**Validation.** The empirical test of whether the receiver can actually consume the state under those assumptions.

216 This distinction avoids a common ambiguity. A contract does not prove that a latent channel works,  
217 just as an API signature does not prove that a function is useful. However, without a contract,  
218 empirical claims are underspecified. A statement such as “we transmit hidden states” does not  
219 identify a communication channel until it specifies the hidden state, the receiver computation, the  
220 layer and position conventions, the adapter or fuser, and the validation distribution.

221 For reporting, the contract should function as a compact interface card for the latent channel. It  
222 should be short enough to include in the main paper and precise enough to reproduce the consumption  
223 path. A reader should be able to tell whether the state is expected to remain usable under a different  
224 checkpoint, neighboring layer, modified position policy, new task family, or only under the reported  
225 setting.

## 226 6 Receiver Readability Validation

227 A receiver consumption contract defines the scope of a latent communication claim. Receiver  
228 readability validation tests whether the receiver’s actual computation uses the transmitted latent state  
229 as task information. This distinction separates the specification of a channel from evidence that the  
230 channel is being used. It is closely related to causal and behavioral validation: a representation should  
231 be evaluated not only by what it encodes, but by whether interventions on it change the model’s  
232 behavior in the intended way [6, 8, 22, 28].

233 The validation problem has a mediation structure. After receiving a latent state, the receiver should  
234 be able to act without direct access to the source information. At the same time, the receiver’s own  
235 context should not already determine the answer. A latent communication claim should therefore rule  
236 out three alternative explanations, summarized in Table 4. This style of validation also follows broader  
237 concerns from shortcut learning and distribution shift: performance gains can reflect unintended cues  
238 or narrow distributional regularities rather than the intended mechanism [2, 9].

239 These nulls make receiver readability operational. Bracketing tests whether the latent channel is  
240 needed at all. Sample-specificity tests whether the content of the transmitted state matters. Scope  
241 robustness tests whether the stated contract accurately describes where the channel remains usable.  
242 Representation metrics such as probing accuracy, hidden-state similarity, or logit KL can support  
243 these tests, but they should not replace behavioral evidence [15, 18, 19].

244 This gives a minimal falsification structure for latent communication. A paper need not run every  
245 possible perturbation, but it should state which null explanations its evidence rules out under the  
246 reported contract. If one family is missing, a specific alternative explanation remains for why the  
247 apparent latent channel may not be communication.

## 7 Alternative Views and Counterarguments

**Existing systems already address receiver consumption.** We do not claim that existing latent-communication systems ignore the receiver. Many successful systems design specific consumption mechanisms, such as adapter-mediated hidden messages, recurrent latent thoughts, attention-side KV memory, or learned cache fusers. Our claim is that these mechanisms remain protocol-specific. Receiver consumption should be treated as the cross-protocol object that explains what such systems share, where their usability claims apply, and how failures should be diagnosed.

**Receiver-consumption contracts may look like a checklist.** The checklist is not the contribution. The main point is that latent communication should be evaluated at the receiver-consumption boundary rather than at the sender-representation boundary. A high-dimensional or probe-decodable state may still fail to influence the receiver’s computation. A contract scopes the usability claim; receiver-readability validation tests it.

**Hidden states can already function as messages.** We agree. The question is not whether hidden states can be messages, but through which receiver interface they are consumed. Residual continuation, adapter-mediated reading, and attention-side memory are different communication objects. Appendix B argues that hidden-state communication is more fragile when raw residual states are injected across models without a receiver-owned read interface.

**End-to-end training may learn the interface.** End-to-end training may learn a useful receiver-consumption mechanism. Our argument is that learned interfaces still need validity scopes. A channel trained on one model pair, layer, position policy, and task distribution may not transfer to another. Contracts make this scope explicit, and receiver-readability tests check whether the interface remains usable under perturbation.

**End-to-end task accuracy may be sufficient.** Task accuracy is necessary, but not sufficient for understanding latent communication. A gain may come from better orchestration, extra computation, additional parameters, retrieval effects, or shortcuts rather than from sample-specific latent content. Receiver-readability isolates the narrower claim: given only the transmitted latent state under a specified interface, can the receiver approach a direct-information baseline while staying above a source-denied baseline?

**Text remains more universal and auditable.** We agree. Text has portability, inspectability, and human audibility that latent states lack. Our position is not that latent channels should replace text. Future systems may use latent channels for high-bandwidth internal computation and text, structured tool calls, or summaries for external commitments and audit. This makes contracts more important: systems should know when a latent state is within scope, when it should be rejected, and when fallback is required.

## 8 Research Agenda: From Protocols to Field Standards

A receiver-consumption view changes what future work should optimize and report. We propose five directions.

**Build receiver-consumption benchmarks.** The field needs benchmarks that isolate receiver consumption rather than only end-to-end system performance. Dual-source latent-prefix tasks are one design: the receiver must answer a query using a latent representation of source information while raw source text is withheld. Other designs may test KV-cache reuse under position shifts, cross-model cache fusion under layer perturbations, or final-hidden-state messages under receiver swap. The common requirement is that the benchmark identifies what information the sender had, what the receiver was allowed to consume, and how close the latent path comes to a direct-information baseline.

**Compare interfaces, not only tensors.** Instead of asking whether “hidden states” or “KV caches” are better in the abstract, future work should compare receiver-consumption interfaces: soft embeddings for compatibility, final hidden states for adapter-mediated messages, residual hidden prefixes for same-model continuation, KV caches for attention memory, projected caches for cross-model



transfer, and recurrent latent thoughts for iterative reasoning. This comparison should report the contract under which each interface works and the perturbations under which it fails.

**Study portability as a first-class problem.** Latent communication will be limited if each channel works only for one checkpoint, layer, position convention, and task distribution. Future work should characterize which receiver-consumption contracts transfer across checkpoints, model sizes, model families, modalities, and decoding contexts. The goal is not to force universal latent messages, but to understand the conditions under which receiver consumption is stable.

**Extend beyond text-only latent agents.** Much current evidence comes from text-only reasoning settings, where natural language is already a strong communication medium. The case for latent communication may be stronger in multimodal, geometric, uncertain, or embodied settings, where the sender state contains continuous perceptual quantities, attention structure, uncertainty distributions, or world-state geometry that are awkward to serialize faithfully into text. These domains should not be treated as afterthoughts; they are where receiver-consumption contracts may matter most.

**Treat audit and fallback as part of the channel.** If latent channels become internal communication layers for agent systems, they should support rejection and fallback. An out-of-contract latent state should not silently steer a receiver toward external actions. Text, tool calls, and structured traces should remain audit layers for commitments, while latent channels may serve high-bandwidth internal computation.

## 9 Scope and Limitations

This paper is a position argument supported by controlled diagnostics, not a comprehensive benchmark of all latent-communication protocols. Our experiments focus on Qwen-family decoder-only language models, residual hidden states, latent prefixes, and selected controlled injection settings. They should not be interpreted as proving that a particular adapter architecture is optimal, nor as showing that all latent states are fragile. They show that receiver consumption can fail in systematic ways that any deployable protocol must specify and test.

The receiver-consumption framework also does not solve interpretability or governance by itself. A contract makes the claim scoped; receiver-readability tests make it empirically checkable; audit and fallback paths make deployment more controllable. None of these makes latent communication inherently transparent. The practical standard should therefore be hybrid: latent channels for high-bandwidth internal computation, and text, structured traces, or tools for external commitments and human inspection.

## 10 Conclusion

Latent communication is motivated by a real limitation of text: language can be discrete, verbose, lossy, and costly as an inter-agent medium. But the token bottleneck is only part of the story. Once agents communicate below text, the next question is whether the receiver can use the transmitted state. We have argued that latent communication has a receiver-consumption bottleneck: the gap between information encoded by the sender and information usable by the receiver’s actual computation path. Existing methods already instantiate several receiver-consumption mechanisms, from soft embeddings and adapter-mediated hidden messages to KV working memories and cache fusers. Their diversity is precisely why the field needs a receiver-consumption framing. Communication should be measured at the receiver, not only at the sender. A latent state is not communication until a receiver can consume it. Future work should therefore specify receiver-consumption contracts, validate receiver-readability under perturbations, build benchmarks that isolate consumption, and design hybrid systems where latent channels support high-bandwidth internal computation while text and tools remain available for audit and external action.

**Acknowledgments.** Omitted for double-blind review.

## References

- [1] Ido Amos, Avi Caciularu, Mor Geva, Amir Globerson, Jonathan Herzig, Lior Shani, and Idan Szpektor. Latent reasoning with supervised thinking states. *arXiv preprint arXiv:2602.08332*, 2026.
- [2] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, pages 2206–2240. PMLR, 2022.
- [4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1285.
- [5] Zhuoyun Du, Runze Wang, Huiyu Bai, Zouying Cao, Xiaoyong Zhu, Yu Cheng, Bo Zheng, Wei Chen, and Haochao Ying. Enabling agents to communicate entirely in latent space. *arXiv preprint arXiv:2511.09149*, 2025. Accepted to ACL 2026.
- [6] Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9:160–175, 2021.
- [7] Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025. Published in ICLR 2026.
- [8] Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. *arXiv preprint arXiv:2106.02997*, 2021.
- [9] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- [10] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.446.
- [11] Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [12] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. *arXiv preprint arXiv:2310.02226*, 2023. Published at ICLR 2024.
- [13] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, pages 3929–3938. PMLR, 2020.
- [14] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. Accepted to COLM 2025.
- [15] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 2733–2743. Association for Computational Linguistics, 2019.

- [16] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.
- [17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [18] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, pages 3519–3529. PMLR, 2019.
- [19] Nikolaus Kriegeskorte, Marieke Mur, and Peter Bandettini. Representational similarity analysis: Connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2:4, 2008. doi: 10.3389/neuro.06.004.2008.
- [20] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.243.
- [21] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 4582–4597. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.acl-long.353.
- [22] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 35, 2022.
- [23] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 487–503. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.eacl-main.39.
- [24] Chau Pham, Boyi Liu, Yingxiang Yang, Zhengyu Chen, Tianyi Liu, Jianbo Yuan, Bryan A. Plummer, Zhaoran Wang, and Hongxia Yang. Let models speak ciphers: Multiagent debate through embeddings. *arXiv preprint arXiv:2310.06272*, 2023. Accepted to ICLR 2024.
- [25] Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [26] Vignav Ramesh and Kenneth Li. Communicating activations between language model agents. *arXiv preprint arXiv:2501.14082*, 2025. ICML 2025.
- [27] Xiangyu Shi, Marco Chiesa, Gerald Q. Maguire Jr., and Dejan Kostic. Kvcomm: Enabling efficient llm communication through selective kv sharing. In *International Conference on Learning Representations*, 2026. Poster.
- [28] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Causal mediation analysis for interpreting neural NLP: The case of gender bias. In *Advances in Neural Information Processing Systems*, volume 33, pages 9573–9585, 2020.
- [29] Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in gpt-2 small. In *International Conference on Learning Representations*, 2023.
- [30] Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot: Soft chain-of-thought for efficient reasoning with llms. *arXiv preprint arXiv:2502.12134*, 2025. ACL 2025 main conference.

- 438 [31] Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot++: Test-time scaling with soft  
439 chain-of-thought reasoning. *arXiv preprint arXiv:2505.11484*, 2025.
- 440 [32] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- 441 [33] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian  
442 Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. Kvcomm: Online cross-context  
443 kv-cache communication for efficient llm-based multi-agent systems. In *Advances in Neural*  
444 *Information Processing Systems*, 2025. Poster.
- 445 [34] Yujia Zheng, Zhuokai Zhao, Zijian Li, Yaqi Xie, Mingze Gao, Lizhu Zhang, and Kun Zhang.  
446 Thought communication in multiagent collaboration. *arXiv preprint arXiv:2510.20733*, 2025.  
447 NeurIPS 2025 Spotlight.
- 448 [35] Jiaru Zou, Xiyuan Yang, Ruizhong Qiu, Gaotang Li, Katherine Tieu, Pan Lu, Ke Shen, Hang-  
449 hang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collabora-  
450 tion in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.

## A Controlled Diagnostic Details

This appendix lists the controlled evidence used in Section 4. The main text intentionally reports only position-level lessons; the numerical diagnostics are summarized here as compact plots. These runs are not cited as external prior work; they are part of the paper’s empirical support. The goal is not to establish a new state-of-the-art method, but to make the receiver-consumption claim falsifiable.

### A.1 Continuation Sensitivity

Same-checkpoint stitching is exact when the hidden state is consumed by the correct next-layer suffix under the same tokenizer, attention mask, position IDs, numerical mode, and decoding settings. Counterfactual continuations break this equivalence, which is the empirical basis for treating residual hidden states as continuation states rather than self-contained messages.

Table 5: Protocol card for the continuation-sensitivity diagnostic.

| Field                      | Specification                                                                                                                                         |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sender / receiver model    | Qwen3-8B used as both sender and receiver under the same checkpoint, tokenizer, numerical mode, and decoding setup.                                   |
| State type                 | Residual hidden state at a decoder cut layer.                                                                                                         |
| Adapter objective          | None; the test isolates exact continuation and counterfactual continuation mismatches.                                                                |
| Consumption variants       | Correct adjacent-layer continuation is compared with replaying the current layer, resuming from a previous layer, and skipping ahead to later layers. |
| Behavioral metric          | Divergence from the full-forward reference after resuming through the receiver suffix.                                                                |
| Main observation           | The hidden state is usable only under the correct next-layer continuation contract.                                                                   |
| Null explanation ruled out | Residual hidden states are self-contained messages that can be consumed independently of the receiver suffix.                                         |

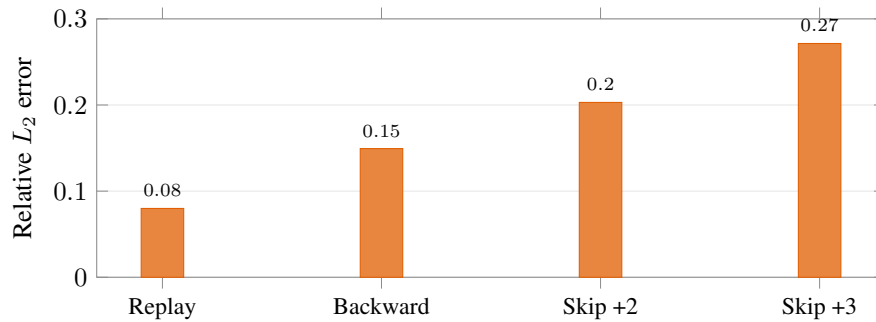


Figure 2: Continuation mismatch increases divergence even inside one checkpoint. Replay feeds the cut state back through the same layer, Backward resumes from an earlier layer, and Skip +2/ +3 resumes too far ahead. Correct adjacent-layer continuation is the uncharted zero-error reference.

### A.2 Cross-Model Compatibility

The cross-model transfer probe maps hidden states from a smaller model to a larger same-family receiver. Training-free dimensional adapters are weak, while hidden-level ridge calibration helps but saturates quickly and remains layer-pair dependent.

Table 6: Protocol card for the cross-model compatibility diagnostic.

| Field                      | Specification                                                                                                                                   |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Sender / receiver model    | Qwen3-8B sender to Qwen3-14B receiver, within the same model family but across width and depth.                                                 |
| State type                 | Residual hidden states mapped from sender width to receiver width.                                                                              |
| Adapter objective          | Compare training-free shape adapters (zero padding, repeat padding, embedding-space least squares) with hidden-level ridge calibration.         |
| Consumption setting        | The mapped sender state is inserted into the target receiver at a selected residual layer, then the Qwen3-14B suffix continues from that layer. |
| Behavioral metric          | Held-out hidden reconstruction and downstream continuation sensitivity under the mapped receiver state.                                         |
| Main observation           | Shape matching is insufficient, and calibrated maps remain layer-pair dependent.                                                                |
| Null explanation ruled out | Cross-model latent consumption follows from dimensional compatibility or embedding-space alignment alone.                                       |

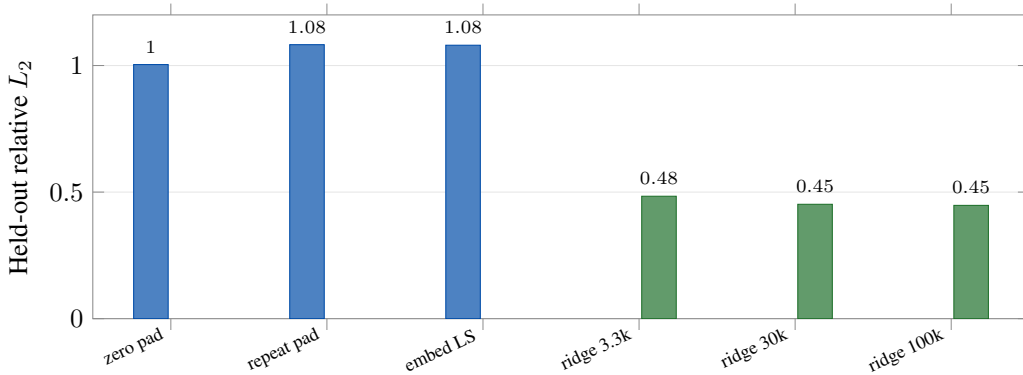


Figure 3: Shape-matching adapters do not make cross-model hidden states consumable. Zero pad appends zeros to match the target width; repeat pad repeats source hidden coordinates until the target width is filled; embed LS is an embedding-space least-squares width map; ridge uses paired hidden states for calibrated linear reconstruction with increasing calibration-token budgets.

### 465 A.3 Alignment Versus Consumption

466 In a hidden-compensation probe, the adapter is supervised to reconstruct the joint-text hidden state.  
467 This improves the representation metric while worsening behavior-level divergences, illustrating why  
468 representation alignment is not by itself a communication claim.

Table 7: Protocol card for the alignment-versus-consumption diagnostic.

| Field                      | Specification                                                                                                                              |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Sender / receiver model    | Same-model Qwen3-8B latent-prefix receiver compared against a direct joint-text reference path.                                            |
| State type                 | Hidden prefix and downstream hidden segment under a compensated latent-prefix path.                                                        |
| Adapter objective          | Supervise an interaction-compensation module to reconstruct the joint-text hidden state.                                                   |
| Consumption setting        | The receiver consumes a latent prefix while raw source text is withheld; the compensated path is compared with the direct joint-text path. |
| Behavioral metric          | Hidden reconstruction error reported alongside segment-level and final-token behavioral divergence.                                        |
| Main observation           | A better hidden reconstruction objective can make behavior-level receiver consumption worse.                                               |
| Null explanation ruled out | Representation alignment, by itself, is sufficient evidence of latent communication.                                                       |

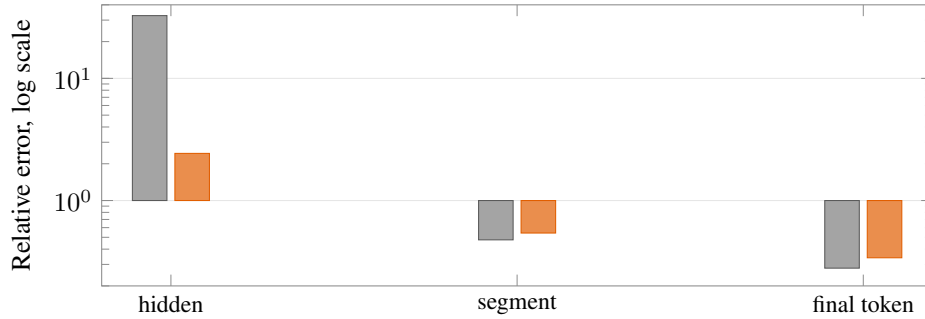


Figure 4: A better hidden reconstruction objective can still worsen the receiver’s downstream behavior. “Before” is the uncompensated latent-prefix path; “after” adds the hidden-compensation module. Hidden is the supervised reconstruction target, while segment and final-token errors measure behavior-level divergence from the direct joint-text path.

#### 469 A.4 Cut-Layer Readability

470 The same-model dual-source latent-prefix setting withholds raw source text from the downstream  
471 branch. The receiver consumes a hidden prefix and then continues through the frozen suffix. The  
472 layer sweep shows that earlier fusion is more readable than later fusion under this contract.

Table 8: Protocol card for the cut-layer readability diagnostic.

| Field                      | Specification                                                                                                                                    |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Sender / receiver model    | Same-model Qwen3-8B dual-source setting with source text withheld from the receiver branch.                                                      |
| State type                 | Source residual hidden prefix inserted at different cut layers.                                                                                  |
| Adapter objective          | No new reader in the layer sweep; vary the receiver consumption layer under frozen-suffix continuation.                                          |
| Consumption setting        | Source and receiver context are run separately to the cut, concatenated as hidden states, and consumed by the frozen suffix from the next layer. |
| Behavioral metric          | Direct-next match and segment-level divergence from the direct-information path.                                                                 |
| Main observation           | Earlier cuts are more receiver-readable than later cuts under this contract.                                                                     |
| Null explanation ruled out | Deeper or more semantic residual states are automatically easier for the receiver to consume.                                                    |

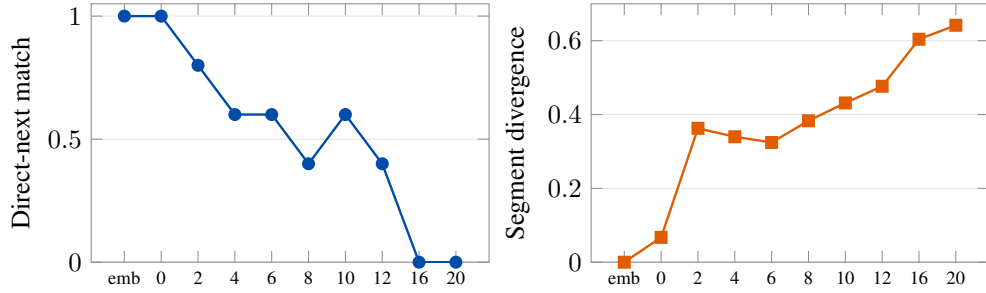


Figure 5: Layer choice changes receiver readability. The x-axis is the consumption cut: “emb” uses the input-embedding interface, while numeric labels are decoder layers after which hidden states are concatenated and the frozen suffix resumes. Early hidden prefixes remain closer to the direct-text path; late prefixes are less composable with the receiver context.

## 473 B Structural Asymmetry in Cross-Model Receiver Consumption

474 This appendix records a structural argument behind several diagnostics in Section 4. The point is  
 475 not that one protocol family must always dominate another, but that different receiver-consumption  
 476 interfaces impose different alignment problems. In particular, cross-model residual-stream injection  
 477 and architectural read interfaces are not symmetric.

478 **Setup.** Let the receiver  $R$  be a decoder-only transformer

$$f_R = \pi_R \circ f_R^L \circ f_R^{L-1} \circ \dots \circ f_R^1 \circ \text{embed}_R,$$

479 where each block has the standard residual form

$$f_R^\ell(h) = \tilde{h} + \text{FFN}_R^\ell(\text{LN}(\tilde{h})), \quad \tilde{h} = h + \text{Attn}_R^\ell(\text{LN}(h)).$$

480 For an attention input  $x$ , write

$$\text{Attn}_R^\ell(x; K, V) = W_O^{R,\ell} \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad Q = W_Q^{R,\ell} x.$$

481 Let  $h_R^\ell$  denote the receiver residual stream after layer  $\ell$ , and let  $\mathcal{D}_R^\ell$  denote the empirical distribution  
 482 of  $h_R^\ell$  on the receiver’s natural data distribution. A sender  $S$  may have different width or depth and  
 483 may expose residual states  $h_S^\ell$  or KV pairs  $(K_S^\ell, V_S^\ell)$ .



484 **Two consumption operations.** In hidden-state consumption, a learned adapter  $\phi$  maps a sender  
 485 residual state into the receiver residual width, and the receiver resumes from layer  $\ell + 1$ :

$$y_R^{\text{hs}} = \pi_R \circ f_R^L \circ \dots \circ f_R^{\ell+1}(\phi(h_S^\ell)).$$

486 In KV-cache consumption, learned projections  $\psi_K, \psi_V$  map sender keys and values into the receiver  
 487 attention interface, while the receiver still runs its own stack. At layer  $\ell$ , attention reads from an  
 488 extended key-value set:

$$\text{Attn}_{R,\text{ext}}^\ell(x; K_S, V_S) = W_O^{R,\ell} \text{softmax}\left(\frac{Q[\psi_K(K_S^\ell); K_R^\ell]^\top}{\sqrt{d_k}}\right) [\psi_V(V_S^\ell); V_R^\ell].$$

489 Both operations perturb the receiver from layer  $\ell$  onward. The difference is the mathematical object  
 490 being aligned.

491 **Layer-specialization assumption.** Assume that each receiver block is smooth on the input dis-  
 492 tribution on which it is normally used. That is, for  $u$  in a local neighborhood of  $\mathcal{D}_R^{\ell-1}$  and small  
 493 perturbations  $\delta$  that keep  $u + \delta$  in that neighborhood,

$$\|f_R^\ell(u + \delta) - f_R^\ell(u)\| \leq L_\ell \|\delta\|.$$

494 No smoothness claim is made once the perturbation moves the residual stream outside this local  
 495 neighborhood. This is the minimal form of layer specialization needed for the argument: layers are  
 496 reliable on their natural residual-stream distribution and uncontrolled off it.

497 **Lemma B.1** (Attention writes through a receiver-owned interface). *For any compatible  $K', V'$  and  
 498 any receiver query  $Q$ , the attention output satisfies*

$$\text{Attn}_R^\ell(\cdot; K', V') \in \text{im}(W_O^{R,\ell}),$$

499 and

$$\|\text{Attn}_R^\ell(\cdot; K', V')\| \leq \|W_O^{R,\ell}\|_{\text{op}} \max_i \|V'_i\|.$$

500 *Proof.* The softmax weights form a stochastic vector, so the pre-output attention value is a convex  
 501 combination of rows of  $V'$ , followed by the receiver's own output projection  $W_O^{R,\ell}$ .  $\square$

502 **Lemma B.2** (Residual injection has no structural validity constraint). *In hidden-state consumption,  
 503  $\phi(h_S^\ell)$  is fed directly to  $f_R^{\ell+1}$ . The receiver architecture supplies no intervening map that projects  
 504 arbitrary points in  $\mathbb{R}^{d_R}$  back onto  $\mathcal{D}_R^\ell$ . Any such validity constraint must be learned entirely by  $\phi$ .*

505 *Proof.* This follows from the definition of the consumption operation:  $\phi(h_S^\ell)$  replaces the residual  
 506 input to the next receiver block before any receiver-owned sublayer can transform it.  $\square$

507 **Proposition B.3** (Alignment-cost asymmetry). *Let the receiver top stack be*

$$T_R^{\ell+1:L} = \pi_R \circ f_R^L \circ \dots \circ f_R^{\ell+1}.$$

508 *For hidden-state consumption, a local bound of the form*

$$\|T_R^{\ell+1:L}(\phi(h_S^\ell)) - T_R^{\ell+1:L}(h_\star^\ell)\| \leq \left(\prod_{m=\ell+1}^L L_m\right) \|\phi(h_S^\ell) - h_\star^\ell\|$$

509 *is available only when both  $\phi(h_S^\ell)$  and the reference state  $h_\star^\ell$  remain in the receiver's local residual-  
 510 stream neighborhood. Off that neighborhood, the layer-specialization assumption gives no bound.*

511 *For KV consumption, the comparable perturbation is the attention-output difference*

$$\Delta_{\text{attn}} = \text{Attn}_{R,\text{ext}}^\ell(x; K_S, V_S) - \text{Attn}_R^\ell(x; K_R, V_R),$$

512 *which lies in  $\text{im}(W_O^{R,\ell})$  by Lemma B.1. When the resulting residual remains in the receiver's local  
 513 neighborhood,*

$$\|y_R^{\text{kv}} - y_R^\star\| \leq \left(\prod_{m=\ell+1}^L L_m\right) \|\Delta_{\text{attn}}\|.$$

514 *Proof.* Both inequalities follow by applying the layer-specialization bound through the receiver top  
515 stack. The hidden-state application is valid only when the injected residual and the reference residual  
516 stay inside the receiver’s local residual-stream neighborhood. For KV consumption, Lemma B.1  
517 first routes the external state through the receiver’s own attention output map, so the perturbation  
518 is measured after the receiver-owned interface has converted it into an attention-output residual  
519 perturbation. The asymmetry is therefore in the domain where the bound is meaningful: hidden-state  
520 injection must land on a usable residual-stream state before the receiver acts on it, whereas KV  
521 consumption passes through a receiver-owned read interface before becoming a residual perturbation.  
522  $\square$

523 **Corollary B.4** (The adapter problems are not equivalent). *KV consumption asks  $\psi_K, \psi_V$  to align*  
524 *sender keys and values with a single receiver attention interface. This is a local interface-alignment*  
525 *problem tied to  $W_Q^{R,\ell}, W_K^{R,\ell}, W_V^{R,\ell}$ , and  $W_O^{R,\ell}$ .*

526 *Hidden-state consumption asks  $\phi$  to produce a residual point that stays on trajectory through the*  
527 *remaining receiver stack:*

$$\phi(h_S^\ell) \in \mathcal{D}_R^\ell \cap \bigcap_{m=\ell}^{L-1} (f_R^{\ell+1:m+1})^{-1}(\mathcal{D}_R^{m+1}).$$

528 *This is a cascaded trajectory-alignment problem in a nonlinear residual-stream geometry. The*  
529 *adapter must satisfy not only a width or representation-matching constraint, but a sequence of*  
530 *downstream receiver-distribution constraints.*

531 **Remark on hidden-state messages.** This argument does not imply that hidden states can never be  
532 useful messages. It says that successful cross-model hidden-state protocols should route the sender  
533 state through an architectural read interface, such as an input-side or adapter-mediated message  
534 interface, rather than directly replacing an intermediate residual stream across models. Interlat-style  
535 protocols are consistent with this view: the relevant object is not merely the hidden state, but the  
536 receiver-side interface through which that state is consumed [5].

537 **Claim B.5.** *Cross-model latent communication should scale best through receiver architectural read*  
538 *interfaces, especially input-side interfaces and attention. Residual hidden-state injection has no third*  
539 *structural pathway: without same-checkpoint continuation or joint end-to-end training, it requires a*  
540 *learned adapter to solve the full residual-trajectory alignment problem.*

541 **Diagnostic results as instances of the asymmetry.** The diagnostics in Appendix A are not isolated  
542 observations. They are projections of the structural asymmetry above: hidden-state residual injection  
543 succeeds only when the receiver trajectory is already valid, and becomes fragile when the consumption  
544 path asks an adapter to create a valid receiver trajectory without architectural support.

Table 9: How the controlled diagnostics instantiate the structural asymmetry.

| Observation                                                                             | Diagnostic | Structural origin                                                                                                                                           |
|-----------------------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Same-checkpoint hidden-state splicing is exact under the correct continuation contract. | A.1        | Sender and receiver are the same computation, so the transferred residual state already lies on the receiver’s natural trajectory.                          |
| Replay, backward, or skipped continuations break equivalence.                           | A.1        | The receiver suffix is no longer the suffix for which the residual state is valid; the local trajectory assumption no longer applies.                       |
| Cross-model hidden-state adapters remain fragile.                                       | A.2        | The adapter must solve a cascaded residual-trajectory alignment problem across the target receiver stack, not just a width-matching problem.                |
| Hidden alignment can improve while behavior worsens.                                    | A.3        | Single-layer representation proximity does not imply downstream trajectory compatibility, and residual injection has no architectural correction step.      |
| Early cut layers are more readable than late cut layers.                                | A.4        | Later residual states are consumed deeper in the receiver trajectory, where the valid neighborhood is more specialized and harder to re-enter by injection. |

545 **What this establishes.** The argument establishes a structural distinction between cross-model  
546 residual injection and architectural-interface consumption. It explains why same-checkpoint contin-  
547 uation, cross-model hidden-state transfer, and cut-layer readability are manifestations of the same  
548 receiver-consumption issue. It does not rule out end-to-end training that changes the receiver, nor does  
549 it provide quantitative error bounds for a particular KV adapter. It is a prediction about scalability of  
550 consumption interfaces, not a theorem about every benchmark regime.